On the Modelling of Aggregated Behaviour for Simulation: An Event–Based Architecture

Adam Banham
Claudia Szabo
adam.banham@adelaide.edu.au
claudia.szabo@adelaide.edu.au
Adelaide University
Adelaide, South Australia, Australia

Ryan Beruldsen
ryan.beruldsen2@defence.gov.au
Defence Science and Technology Group, Department of Defence
Adelaide, South Australia, Australia

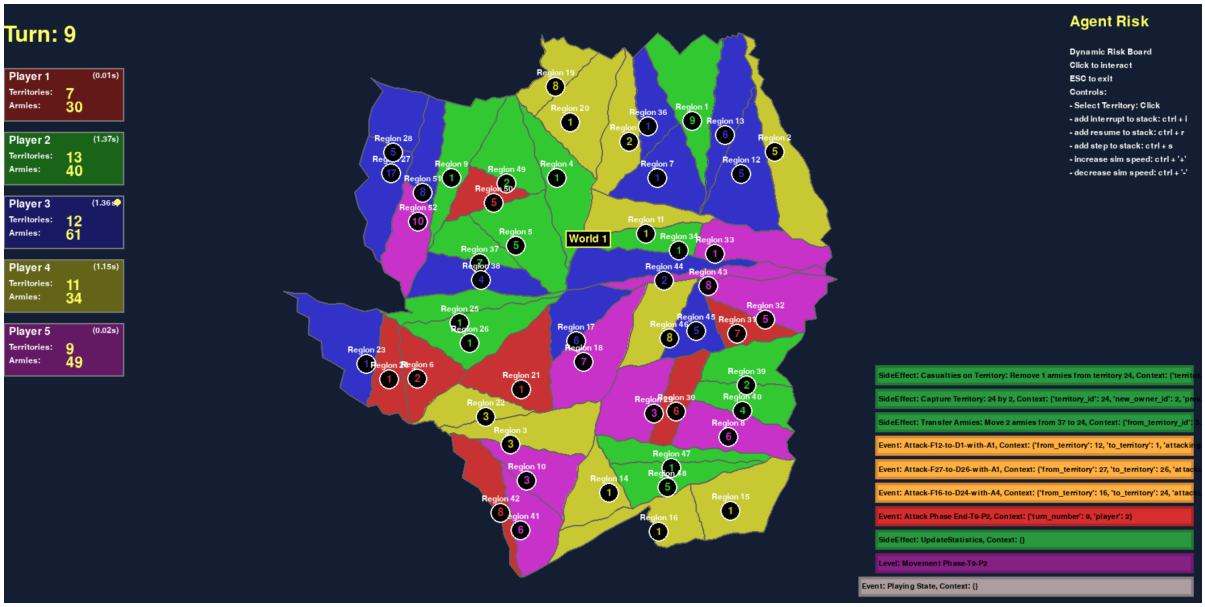


Figure 1: A screenshot of the simulator using our proposed event-driven architecture to simulate a game of Risk.

Abstract

Studying complex systems often requires simulation to confirm or test hypotheses about their behaviour. Drawing insights from simulation studies can empower a large domain of applications, but only if realistic behaviour can be modelled. Simulators are often bespoke and architectured without considering the long-term generalisation for simulation studies. Furthermore, when studying complex systems, both modellers and simulation engineers need to support the modelling of systems where emergent, nonlinear, or self-organising behaviour may occur. In this paper, we identify several modelling and simulation considerations that influence the simulation life-cycle within a large-scale discrete event simulator. We compare the suitability of various notations and formalisms against these considerations, and advocate for aggregated behaviour to develop and operationalise macro-level behaviour. Through a novel simulation architecture, we investigate the capabilities of several notations to address our proposed considerations and their runtime performance.

SIGSIM-PADS '26, Vienna, Austria

ACM ISBN 978-1-4503-XXXX-X/2026/06
https://doi.org/XXXXXXX.XXXXXXX

CCS Concepts

• **Computer systems organization** → **High-level language architectures**; • **Software and its engineering** → **Abstraction, modeling and modularity**; • **Computing methodologies** → **modeling and simulation**.

Keywords

Simulation, Agent Behaviour, Aggregated Behaviour, Planning, Software Architecture

ACM Reference Format:
Adam Banham, Claudia Szabo, and Ryan Beruldsen. 2026. On the Modelling of Aggregated Behaviour for Simulation: An Event–Based Architecture. In *Proceedings of ACM SIGSIM International Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS '26).* ACM, New York, NY, USA, 13 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Complex systems are ubiquitous in today's world [39, 55, 55, 61, 62, 67, 91], and their study requires the modelling and analysis of their components, the environment, and the interactions between many agents to be modelled, simulated, and visualised [84]. Complex systems comprise many intricately connected components with one or more levels of feedback, producing emergent [72] or nonlinear behaviours, self-organisation, and other unexpected behaviours[39, 84]. Understanding emergent behaviour in complex systems has many applications across many domains, such as businesses [27, 100], biology [72], healthcare [2, 3, 47], and military domains [93, 105], where insights from the simulation studies are used to inform policies with immediate critical effects.

A critical challenge in the modelling, simulation, visualisation, and analysis of complex systems stems from the lack of understanding of the relationships between *macro level*, system-wide behaviours, and *micro level* behaviours at the component or agent levels [10, 15, 35, 44, 77, 89]. Both approaches to defining behaviour can be error-prone due to the modeller's design choices, and each has its own advantages and disadvantages, as well as having many options for a supporting notation [1, 18, 20, 21, 54, 60, 78, 79, 107] to design behaviour. The choice between notations is left to the subject matter expert or the modeller and can influence the feasibility of the simulation study and the trustworthiness of its results [96]. In addition, without a rich contextual layer, the composition and reuse of designed behaviours is very limited [96], leading to bespoke, ad-hoc simulations.

Further challenges arise from the approaches used to model and simulate complex systems, which typically employ either a top-down or a bottom-up approach [72, 93]. In a *top-down* approach, the focus is on the engineering of a desired system behaviour, which is achieved through the interactions between purposely-designed system components that lead to that specific macro behaviour [56]. Challenges include deriving micro behaviour rules and interactions between components from that desired high-level behaviour [87]. The modelling process itself might pose challenges as modellers focus on adding fidelity to a behaviour [93] as needed to increase the detail of modelled interaction. For instance, in military simulations [93], large-scale battles may use aggregated entity models like attrition models [31, 46], and simulators might only use more detailed interaction-based models for smaller-scale duels [9]. In addition, the modelling of the desired macro-level behaviour is explicit, but micro-level behaviours and interactions will not necessarily be so, due to many confounding factors such as available subject matter expert time and resources, or the lack of specific formalisms to connect macro to micro level behaviours [57, 92].

In contrast, a *bottom-up* approach focuses on defining micro-level components and their interactions, and adds macro-level behaviours as needed to aggregate behaviour or interactions. One such example is bird flock models [72], where birds are first modelled as individuals, and then an aggregated interaction (a "bird-oid" or *boids* object) is introduced, allowing for the emergence of flocks. Other case studies can be observed within ecology [24, 82], such as schools of fish [68]. For bottom-up simulations, the aggregated interaction can be iteratively built and extended until satisfactory. Challenges in this approach include deriving a macro, system-wide behaviour from the interactions of micro-level behaviours, and the verification and validation of new behaviours that emerge. A critical challenge for both approaches remains the effective modelling of interactions between components/agent [15, 88] and understanding their effects on other components or the environment [13, 58, 86].

To overcome these challenges, the formalism or notation used to model complex systems and their components becomes critical [96]. Furthermore, selecting a notation that is expressive enough for future simulation studies is crucial to their success. An ideal notation would support the design of behaviour that allows for both bottom-up and top-down modelling approaches. Furthermore, the notation should be flexible enough to accommodate future studies and enable rapid reuse of previously designed behaviour. This paper focuses on selecting and highlighting notations to build a rich, reusable modelling and simulation approach for encoding complex behaviours, responding to recent calls for modelling advancements [96].

This paper contributes several intermediate outcomes that build towards a holistic simulation approach for complex systems. We present a series of design considerations that capture critical perspectives of modelling and simulation, informed by extensive discussions with software engineers who employ a large component-based discrete event simulator. We consider several notations from simulation [40, 60, 107], process mining [1, 54, 100], and software engineering [79, 83] communities, discussing their capacity to address these concerns and operationalise the methodology. We propose an interaction-focused simulation architecture that uses these notations to model agent interactions and their higher-level strategies. Lastly, we instantiate our architecture through a prototype simulator (demonstrated in Figure 1) for the board game 'Risk', and use the prototype to evaluate possible notations for the architecture. Concretely, our contributions are:

- A series of modelling and simulation considerations for simulating complex systems;
- A comparison of possible notations for modellers and engineers simulating complex systems;
- An interaction-focused simulation architecture for simulating complex systems; and
- A prototype of the architecture and evaluation of the possible notations' capacity to model behaviour.

2 Background and Related Work

This section highlights existing literature on modelling notations, simulation approaches, and visualisations for complex behaviour from various research domains, namely, modelling and simulation, game development, and business process management.

The discrete event system specification (DEVS) [16, 107] is a mathematical formalism that captures the behaviours of components operating in a discrete event simulator. A DEVS system encapsulates a collection of models that are either atomic or coupled. Coupled models aggregate networks of atomic and other coupled models, enabling both decomposition and composition for modellers. Communication between elements is established via input and output ports using events. An atomic or couple model describes dynamic behaviour based on internal and external events. Internal events are used to denote how a model unfolds over time through some discrete states. External events are observed through the input

ports of the model, and may trigger changes within the model as they arrive. Models may place values in output ports before undertaking an internal transition between states. The strong mathematical formalism behind DEVS has been shown to have many benefits and applications for simulation studies [48, 63, 64, 73, 101, 102]. Notably, DEVS should be seen as the *glue* of a simulator, and may not help the modeller find an appropriate modelling representation for a system [41]. Thus, out of the box, DEVS does not provide constructs for common workflow behavioural patterns [98], leaving their construction to the modeller.

The use cases for *behaviour trees* (BTs) in game development, particularly for Non-Playing Characters (NPC)s behaviour design, are well known [40, 71]. Behaviour trees can be seen as AND–OR trees that store all possible plans that an agent can take, composed of high-level goals that decompose into lower-level goals until they reach an action for the agent [70]. Alongside the adoption in industry, the notation for behaviour trees has been refined and developed [43, 52, 78], highlighting that BTs has great expressiveness for explicit planning. One key downside is the usage of "blackboards" as the source for feeding data or contextual data into the semantics of these trees, which can lead to complex data races [78]. However, a major architectural benefit of behaviour trees is their modularity and composability, which allows for swapping a leaf node with another behaviour by adding a *sub-tree* [17]. Meaning modellers can set up idoms or common patterns beforehand and reuse them while building out contextual agents for their simulations.

A more implicit notation for constructing plans over behaviour is hierarchical task networks (HTNs), which focus on creating a plan from a planning domain consisting of actions [33, 60, 75, 80]. Tasks are recursively defined until the most bottom-level recursive task returns a primitive task that can be performed directly using the planning operators [34, 60]. Seminal work on HTN [34, 60] argues that they better characterise the *way users think about problems* by abstracting away from the plan's implementation and instead focuses on the actions needed to achieve a goal. By focusing on goals and encoding them into an HTN, this approach has enabled many applications across several domains [33], such as bridge games [80], real-time strategy games [65], Robotics [37], and military operation simulations [59]. In comparison to BTs, HTNs do not have execution semantics, nor are they directed graphs; therefore, modellers need to carefully consider when to trigger planning or replanning in real-time settings.

While the previous notations are commonly seen within game development for NPC and multi-agent simulations, the following notations come from a different background, business process management [27, 100]. These notations have helped organisations around the world to orchestrate millions of concurrent process instances [32, 74] and capture concurrent behaviour [1, 54, 104]. We explore these notations as potential alternatives for representing complex behaviour to non-technical domain experts. As in the field of business process management, these process models have successfully bridged the gap between implementors and process owners [50, 66].

To capture the control-flow of data-driven decision making in processes, the Decision Model and Notation (DMN) [8, 22, 23, 36, 49] can be used to capture the decision logic between input and output variables for a moment in a process [8, 49]. In simulation studies, DMN could be used to model how agents' internal states are determined or how the world state changes within the simulator. Often, DMN is combined with the business process model and notation (BPMN) to model processes in a holistic manner, and recent work has focused on the execution semantics of combining these notations [21]. DMN has a rich and robust specification that provides precise semantics across applications [54], but, to the best of our knowledge, it has not been used directly in simulation studies.

As DMN focuses on representing the flow within decision logic, it needs to be combined with Business Process Model and Notation [1, 27] (BPMN) to describe control-flow of a process, e.g. first the agent walks to the truck, then they buy ice-cream, after which they can eat the ice-cream. BPMN has many control-flow constructs for branching paths [103] (XOR, OR, Event-driven), aggregation (sub-process tasks), and interrupting events (both catching and throwing intermediate events). Constructing a tractable process model in BPMN that uses all of the expressive power of BPMN is often not the best approach [27, 53]. Furthermore, the extensive expressiveness of BPMN does pose challenges for simulation [99]. Nonetheless, many options exist for simulating processes modelled in BPMN [14, 25], albeit with their own caveats.

An alternative approach to modelling processes is to use Data Petri nets (DPNs) [20], which combine Petri nets with data execution semantics to capture runtime constraints on processes. This notation is far less expressive than BPMN, but it offers more precise execution semantics, enabling complex verification techniques and compliance checks before the model is executed [5]. DPNs introduce data constraints to runs of the Petri net by allowing transitions to have *guards*, which evaluate whether an enabled transition *should* fire [20]. Many verification techniques exist for DPNs for several guard representations: FEEL-formed guards [4, 7], guards with arithmetic [29], and guards with relations between variables [28]. Furthermore, semantic inconsistencies within DPNs can be automatically repaired using many techniques [30, 106], thereby empowering modellers to focus on design rather than runtime concerns. However, Petri nets can suffer from a state-space explosion when dealing with highly concurrent and interleaved behaviour, which may make models overly complex for stakeholders.

A benefit of BPMN, DMN, and DPNs over some other notations is that these conceptual models can support a concise representation of highly concurrent behaviour, meaning that models can have an infinite language[1] using a finite number of modelling elements. This could be seen as a double-edged sword for designing agents in more stringent simulations, such as military applications [105].

An interesting alternative approach to conceptual modelling is Monte Carlo Tree Search (MCTS), which has seen recent breakthroughs in handling complex decision making in the development of realistic NPCs [69, 76, 79]. MCTS is an algorithm for exploring a state space without needing to encode behavioural constructs, offering efficiencies for modellers by only modelling actions and decoupling their logic from the world state [11, 83]. The MCTS algorithm iteratively builds a state space within a given computational budget, identifying the most rewarding next action. The iteration consists of four phases: **selection** finds a leaf of the tree to expand; **expansion** of the leaf into a node with children via some

[1]The language of these models is the set of possible executions of the model.

action; **simulation** to produce an outcome for the expanded node; and finally, **backpropagation** occurs to feed back a reward from the expanded leaf to the root. MCTS has been successfully used in several domains to improve decision making of autonomous agents, such as playing GO [51, 79], military operations [105], game AI development [69], and autonomous vehicle management [38, 45]. For modellers, MCTS enables complex decision-making for agents, provided that a metaheuristic can be defined to guide meaningful exploration of the state space.

2.1 Simulating Aggregated Behaviours

We now introduce a taxonomy of behaviour in simulation models to position the typical modelling and simulation life cycle for modellers working with large discrete event simulators. This taxonomy was informed by our observations of how subject matter experts construct, use, and reuse models while performing simulation studies of complex systems.

The simulation life-cycle consists of several phases, described extensively in the literature Tolk [93, chapter 5], Banks [6, chapter 1], Davis et al. [19], and Uhrmacher and Weyns [97, Chapter 5]. In this paper, we focus on conceptualisation and scenario construction and execution and explore relevant considerations in Sec. 3. In particular, our research direction focuses on supporting modellers in building their models by focusing on agent behaviours and on how these behaviours can be aggregated to build more complex behaviours. To fully develop this idea, we now present a behaviour taxonomy that builds on primitive and compound behaviour typically seen in BTs [40, 70, 78] and HTNs [33, 34, 60] literature towards defining our *aggregated* behaviour.

2.1.1 Primitive Behaviour. From the modeller's perspective, primitive behaviours are simple subroutines that can trigger changes in the simulated world. Primitive behaviours represent the base modelling unit for all behaviours for modellers in our taxonomy.

For instance, in a real-time strategy domain, the pool might consist of $\{\texttt{move}(e,p), \texttt{attack}(e,p), \texttt{sense}(e,d)\}$, to represent that entities e can move to a point p, attack towards a point, or sense in a particular direction d.

2.1.2 Compound Behaviour. Primitive behaviour can be orchestrated into more complex behaviour, requiring abstractions over an ordered set of parameterised primitives. These abstractions are usually operators for idioms, like sequence $\oplus_{\rightarrow}$, choice $\oplus_{\times}$, repetition $\oplus_{\circlearrowleft}$, or concurrency $\oplus_{||}$. Furthermore, compound behaviour is recursive and can be composed of compound and parameterised primitives. Once instantiated, the sequence of primitive and compound behaviours cannot be changed and the parameters of all behaviours are fixed.

For instance, the modeller might design a $\texttt{route}(\langle\langle e_1, \cdots, e_n\rangle, p\rangle)$ compound behaviour taking a collection of entities $e_1, \cdots, e_n$ and moves them to a point p. This compounding of behaviour could consist of $\texttt{route} \equiv \oplus_{||}(\langle\langle\cdots\rangle, \oplus_{\circlearrowleft}(\oplus_{\rightarrow}(\texttt{sense}(e', p), \texttt{move}(e', p))\rangle)$, whereby each entity e' is concurrently sensing and moving towards p until some condition stops the repitition $\oplus_{\circlearrowleft}$. Notably, the expansion of these operators would occur on the agent side, and the resulting expansion would then be passed to the simulator with fixed parameters in an organised manner. In this sense, compound behaviour is static, in that once the sequence and configuration of its compound and primitive behaviours is established, it cannot be changed. The use of compound behaviours mimics a frequent modelling approach that we observed during our interactions with subject matter experts (SMEs), where modellers are focused on developing simulation assets quickly, without consideration of further reuse [90, 95, 96].

2.1.3 Aggregated Behaviour. As discussed, a downside of compound behaviour is that once the compound behaviour is instantiated, it cannot be changed. In addition, the parameters of the compound behaviour need to be known before it is instantiated. This prevents them from being useful in partial visibility systems or in systems where the configuration of a behaviour is dependent on the current system state. Aggregated behaviour seeks to address this drawback by introducing an interaction life-cycle into the orchestration of compound behaviours, allowing parameters to be adjusted based on the current context.

For instance, the modeller might want to model an aggregated behaviour called $\texttt{rush}(\langle\langle e_1, \cdots, e_n\rangle, p\rangle)$, whereby entities $e_1, \cdots, e_n$ are pushing towards a point p. However, the engagement might be far more dangerous than expected, so at some point, they want to model when the push should stop. The simulator controls the individual outcomes of a requested event; so the modeller needs a choice based on what comes after an event. The cycle of request ϕ_r, acceptance ϕ_a, consequence ϕ_c, and finishing ϕ_f represents an interaction life-cycle. Leading to an aggregated behaviour of $\texttt{rush} \equiv \oplus_{\times}(\langle(\phi_{win}, \oplus_{||}(\langle\cdots\rangle, \texttt{attack}(e', p))), (\phi_{lose}, \oplus_{||}(\langle\cdots\rangle, \texttt{move}(e', r)))\rangle)$. Whereas if the last push was winning the engagement ϕ_{win}, the alive entities continue to rush, but if the previous push was lost ϕ_{lose}, then the remaining move back to safety r.

A crucial element of aggregated behaviour is that these interaction life-cycles must be modelled and made available at design time to enable the construction of such behaviour. Similar to compound behaviour, nothing prevents these life-cycles from being connected through a chain of reactions or being recursively constructed.

3 Design Considerations

This section outlines design considerations for practitioners and modellers when simulating large-scale complex systems with many components and elaborate interactions. These considerations are critical to address at the design-time of a simulation architecture, as changes become costly and time-consuming to revise later.

We followed an iterative process in extracting these considerations, both from the simulation literature [84, 93, 96, 107] and through a series of workshops with subject matter experts. Our SMEs were simulation engineers and/or software developers who built large-scale simulation scenarios in an existing large-scale modelling and simulation system. Being able to successfully model a complex system in the selected modelling and simulation environment requires extensive programming background (so thus not necessarily suitable for simulation engineers), as well as deep subject matter expertise (so thus not ideal for software developers). Our synthesis was derived from a series of focus groups aimed at understanding the system's end goal and observations of SMEs' interactions with the system. We iteratively validated the proposed considerations through a series of workshops with all SMEs, and

grouped them into three categories: *notation*, *simulation*, and *mixed* considerations. We summarise the comparison between the formalisms and notations described above in Table 1 and expand the discussion below.

3.1 Notation Considerations

This category focuses on the importance of formalisms and notations to the modelling process. Without a rich notation for the descriptive and operational aspects that facilitate agent decision making, modellers will struggle to develop complex behaviour as outlined in Sec. 2.1. We outline several design considerations to facilitate the choice of notation. Firstly, the notation should support **internal state** (IS), meaning it supports a data state to be internally used for the decision logic model instances. For example, an execution of a DPN [20] would use the running data state of that execution as the internal state of the agent. Similarly, instances of BTs have "blackboards" [40, 78] to capture the current state of their explicit plan.

In addition, notation should support **encapsulation** (CAP), meaning that it is possible to express that an element is an instance of the notation or to support a recursive relationship. For instance, a BPMN model can contain a task that is a *sub-process* task, allowing a single model to be composed of other models. Similarly, BTs support composability through sub-tree substitution without affecting the execution semantics in a classical sense [17, 40]. However, without some change in the execution semantics, a planning domain for HTNs should be concrete and cannot contain another instantiation of a HTN [34]. Likewise, classical Petri nets [100] cannot express that a transition or place represents the execution of another net.

Notation should also support **reactivity** (RCT), meaning that it allows for the interruption of an execution to be halted until an external stimulus occurs, usually represented as a routing construct. Reactivity is essential for modellers to consider if agents are required to have contextual and situational awareness of the world they perceive [85]. For instance, BTs in the Unity game engine allow for interrupts to cancel actions across child nodes at a similar depth and jump to another. Likewise, BPMN models support event-driven gateways [1] to create a state of the execution that waits for external events before continuing. Planning techniques like MCTS [11, 83] and HTNs [34, 60] struggle with this consideration without complex replanning, monitoring, and explicit implementation support.

Notations that can be **transformed** (TFM) into bisimilar notation afford modellers opportunities to model behaviours in higher-level abstractions, and have guarantees about behaviour through formal verification techniques [12, 42]. For instance, BPMN has many control-flow constructs, i.e. OR-gateways [103], which can make verification intractable due to the expressiveness of the constructs. However, restricting the BPMN model to a subset of constructs allows for reconstruction of the model as a coloured Petri Net via prototypes [26]. Similarly, Petri nets are often translated into transition systems [100] to verify behaviour like soundness or liveness. We posit that whether a bisimilar transformation exists for MCTS, HTNs, and BTs depends on their specific implementations.

In practice, any notation can be formally useful but rejected by real-world modellers and stakeholders [50, 53, 81, 99]; thus, considering the overall **approachability** or **ease-of-use** (EoU) of the notation is essential. Ideally, the notation should be equally usable by experts, non-software engineers, or non-experts to build and design behaviours. Furthermore, to effectively communicate simulation results, the notation needs to be understandable to stakeholders; otherwise, the simulator may come into question, eroding overall confidence in the accuracy of the results [96]. One approach could be to consider graph understandability for the notation, which quickly diminishes as the number of nodes and the semantic complexity of those nodes increase. Some say that 50 elements is the turning point of graph understandability [53], though not all considered notations have a straightforward graph to consider.

A further consideration is **visual representation** (VRP), meaning that the notation should support the visualisation of a simulation run from an initial state to a partially completed state, either in terms of control-flow or data-flow. The notation should have an agreed-upon visual representation with clear visual semantics, such as DMN [54], BPMN [1], or UML [18]. If a notation has both a visual representation and precise execution semantics, then it supports animation, which has been established as an effective tool for communication with stakeholders [96].

3.2 Simulation Considerations

These considerations are essential for simulation engineers to construct a simulation that supports the rich analysis of complex systems.

Simulators need to describe states and thus require **world state** (WS), meaning they support describing a simulation state and its evolution between changes. For instance, a simulator built on top of DMN could support the decision logic for transforming an input state into a new state. Most of the other notations are about the planning or execution of actions, rather than encoding data constraints.

To simulate complex systems, simulators should support **interactions** (INT), in which events are treated as an unfolding chain reaction from an originating event. For instance, an agent may request that an event be actioned by the simulator; the simulator then decides whether that originating event is valid; in doing so, the simulator broadcasts that decision (accepted/rejected), triggering further reaction. Within an aggregated behaviour, the agent from where the interaction originates can make use of this resulting decision to inform future events that pertain to the interaction. Interactions should be described through a discrete transition system or through a *life-cycle* that requires, at a minimum, start, intermediate, and end events. Notations with execution semantics that support waiting, such as BPMN, BT, and DEVS, are well-positioned to address this consideration, whereas HTN and MCTS would struggle.

Simulation studies are often repeated and rerun with adjusted parameters, so simulators should enhance **scalability** (SCA), by supporting numerous concurrent agents, or infinitely many subgroups over a set of finite agents for aggregated behaviour. For instance, a simulator built on BTs would rely on "blackboards" to communicate data changes, which could easily lead to scalability issues due to data races. But this issue has not stopped BTs from being applied to many real-time strategy games [71]. In contrast, many process orchestration engines have demonstrated that BPMN models can handle processing millions of process executions in ERP

Table 1: Comparison of notations across our concerns. Rather than considering a binary outcome for support, we consider low support , general support ↑ , and strong support ↑↑ . Items are ordered by the number of ticks allocated.

	Notation						Simulation				Mixed					
Notation	IS	CAP	RCT	TFM	EoU	VRP	WS	INT	SCA	CON	TCT	VER	IPEC	TEMP	ADPT	EXT
DEVS (18)	↑	↑↑	↑	↑		↑		↑	↑↑			↑↑	↑↑	↑↑	↑	↑↑
BTs (18)	↑	↑↑	↑		↑	↑↑		↑	↑↑		↑↑		↑↑	↑↑	↑	↑
HTNs (4)		↑							↑		↑		↑			
DMN (9)						↑↑	↑↑			↑↑		↑	↑↑			
BPMN (22)	↑	↑↑	↑↑	↑	↑↑	↑↑		↑↑	↑↑		↑↑		↑↑	↑	↑↑	↑
DPN (16)	↑			↑↑	↑	↑	↑		↑	↑	↑↑	↑↑	↑↑			↑↑
MCTS (4)					↑				↑		↑		↑			

systems worldwide [32, 74]. However, these executions typically do not have a mutable world state.

Simulators should also support **consistency** (CON), meaning that the simulator allows for the consistent representation of truth in all participating sub-systems, which supports composability [94]. Support for this concern must be factored into the simulator's architecture, not after the fact [94] as reengineering a simulator for composability requires significant changes. Without foresight into the simulator's potential for maintaining this consistency, it will become bespoke, requiring endless modifications for each new scenario description. For instance, DEVS models are intended to model distributed systems rather than provide a consistent truth across subsystems. Meanwhile, DMN focuses on describing the data flow from inputs to outputs to ensure a consistent representation of data outcomes.

3.3 Mixed Considerations

Some considerations span both the domains described above. For example, both notation and simulator should support **tractable executions** (TCT), meaning the execution of a concrete plan in a notation should be able to produce structured loggable event mutations, actions, and events in the simulator. For instance, an execution of a conceptual model from its start to its eventual closure should be translatable into structured data such as eXtensible Event Stream or *XES* [104], for post-simulation processing. This execution should be directly replayable on the conceptual model, allowing a repeat of the simulation and animation.

For simulation studies to be **verified** and **validated** (VER), they require that the notation and simulator allow for their behaviour to be verified and validated against agreed-upon criteria. For instance, an agent cannot move if it has been defined as 'imobile' unless other agents pick it up. Similarly, to compare different instantiations of notations that exhibit similar behaviour, we need to verify that both agents produce the same behaviour in a specified state. Notations with strong formalisation backing, such as DEVS and DPN, strongly support this concern, as they offer many verification techniques.

To diagnose logic problems in simulations, the notation and simulator should support **introspection** (IPEC), meaning that their states are analysable to understand why further steps were taken and what the next possible steps were. Furthermore, like an IDE debugging session, their combination should be expressive enough to allow for the options to *step into*, *step out*, *step over*, or *step back* through states. Notably, all considered notations are 'white box' methods, meaning tooling can be built to introspect their states.

In many simulations, actions take time, so we need **temporality** (TEMP), meaning that the notation can express that tasks/actions might take time, and the simulator can handle events that unfold over a duration rather than discretely. Not only do tasks need to have durations, but their actions also need to be reversed through the simulation run. For instance, in DPNs, transitions (actions of these models) are fired instantly, which poses challenges for this consideration. Meanwhile, BTs, DEVS, and BPMN provide direct support for waiting to represent a duration.

When simulating partially observable systems, behaviour needs to be **adaptivity** (ADPT), meaning that when interactions occur in the simulator, the outcomes can be passed to an unfolding plan in the notation, thereby adapting the next step produced by the plan to the current context. Without support on both sides, a bespoke pipeline would be needed to manage the interaction between aggregated behaviour and the unfolding simulation, affecting verification concerns due to bespoke implementation design choices. For instance, in BPMN, a combination of message flows and boundary events could be used to capture what happens when interactions occur as an agent's plan unfolds. Similarly, BTs could utilise the ticking system to capture that an external rejection has happened for a node that is reporting 'running'.

Finally, both the notation and simulator should be **extendable** (EXT), where they should be easily extended with new execution semantics. It is almost impossible to know all the behaviour constructs [98] that will be needed in future scenarios, so the notation needs to be flexible enough to introduce extensions to induce new semantics. Meanwhile, the simulator needs ways to enact these extensions without rewriting the simulation architecture. A rigid notation for execution, like within BTs, may struggle with supporting this property. While Petri net-based notations can be extended with new semantics in a relatively straightforward manner, as seen in the large family of notations constructed from Petri nets [20, 25, 26, 100].

3.4 Discussion

We summarise our synthesis of considerations for several notations in Table 1. A notation was said to provide strong support (↑↑) if the notation has a directly supported construct for the consideration. If a notation does not have a direct construct, but some complex

automaton within the notation could, then we said it provides general support (↑). Otherwise, limited support () is reported.

We acknowledge that a single notation is unlikely to address all considerations. For example, if a notation were expressive enough to address all besides VER and EoU, it would struggle to address ease-of-use due to the complexity of the notation confusing modellers or domain-level experts. Also, any notation with such complexity is likely to induce a state space that is intractable and thus could not support verification of its execution.

Our synthesis in Table 1 indicates that the following notations form the top group across our considerations: DEVS (18), BTs (18), BPMN (22), and DPN (16). A key difference of notations within this group was that execution semantics were essential. With execution semantics, the notation is sufficiently expressive to construct behaviours that are adaptive and reactive to interactions within the simulator. Opposed to the simulator having to build a complex pipeline to enable adaptivity and reactivity, as one would for MCTS, HTNs, DMNs, and DPNs.

Similarly, if the notation lacks the expressivity to capture temporal considerations, the simulator needs to build a pipeline to support durations for actions. The notations in the top group all provide ways to denote that an execution is waiting, so the simulator can simply pass events along in a straightforward manner. Temporal and external events are also critical for modelling and enacting aggregated behaviour in our setting.

Another characteristic of the notations in the top group was that they were extendable. The ability to introduce new semantics into the notation provides many affordances for modellers and engineers. Whereby, they can create custom notation for their behaviours in simulations without overhauling the entire simulator pipeline.

4 Proposed Architecture

We now propose a novel software architecture (Figure 2a) that embodies the above considerations (Sec. 3), while enabling the design and enactment of aggregated behaviour (Sec. 2.1.3). We demonstrate our architecture through the simulation of a game of Risk where players are automated agents. Our proposed architecture comprises several modules that govern the evolution of the complex system's state and record unfolding interactions within the simulator. Stacks and tapes record the current state of unresolved events and a history of resolved events, respectively. Engines capture aspects of a simulation through responses to events, which in turn produce new events for the simulation. Side effects are a special type of events that encode changes within the world state. Finally, we demonstrate how these elements come together to simulate a game of Risk.

4.1 Stacks and Tapes

Our stacks and tapes approach supports interactions (INT) within the architecture, whereby the simulation unfolds as a chain reaction of events. We represented the chain reaction as an '*event stack*' as shown in Figure 2b and Figure 3. The event stack records the currently unresolved events in the reaction, with the top of the stack representing the next event in the chain. If the event stack becomes empty, the reaction halts, and the simulation ends. For instance, each simulation of Risk starts with a stack consisting of a single event, '*game*'[2]. To encode primitive behaviours or to orchestrate more complex behaviour, engines (discussed later) observe the event stack and produce events when *game* is popped off the stack, continuing the reaction.

To represent the world state (WS) and record tractable simulation executions (TCT) we introduce a second abstraction, *event tapes*. In comparison to stacks, tapes start empty and can only grow to record a history of events that have been resolved. A tape consists of a stack of events, but when read from bottom to top, the events describe what has happened within a simulation. In Figure 2a, we show an event tape recording how an agent's turn could unfold. The tape enables the recovery of a simulation by stepping through it until a desired point is reached, allowing the event stack at that moment to be recovered as well. Tapes create opportunities for listeners of the simulation to have a world state without tightly coupling them into the unfolding simulation using the observer pattern. Furthermore, tapes can serve as an intermediate language to support tooling efforts without strict limitations on their implementation.

4.2 Engines

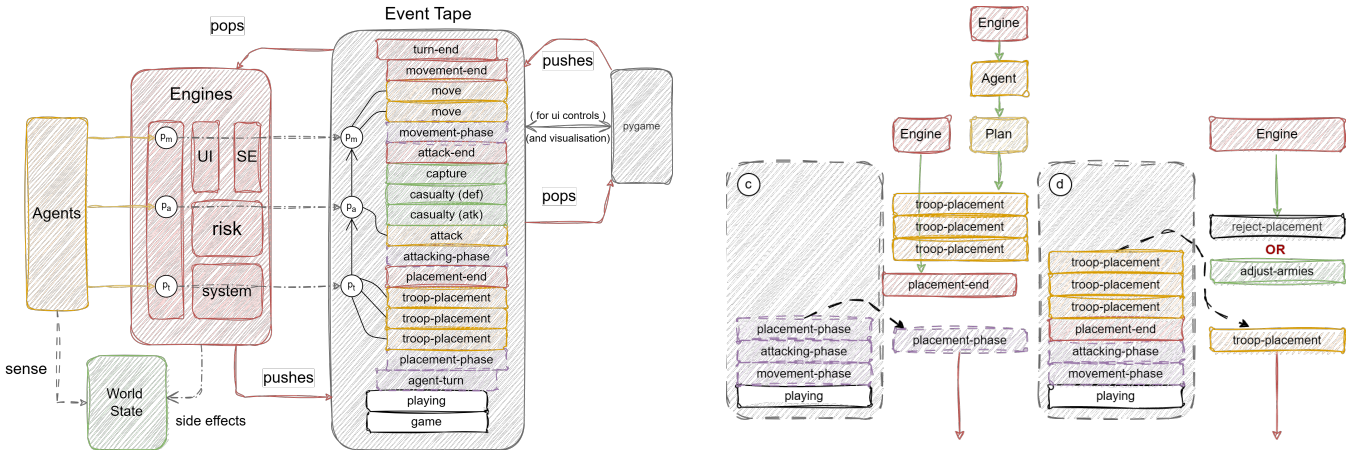
In comparison to event stacks, which capture the state of the chain reaction, *engines* act as the external stimuli for the reaction, rounding off our consideration of interactions (INT). An engine captures how and when reactions occur within the simulation in response to events. In Figure 2b, we showcase how popped events are interactable via engines in our architecture. The figure illustrates the evolution of a simulation over a short period, where a single event is popped, after which engines can respond. Engines listen for a subset of events and, in response, run a deterministic subroutine to produce events that are pushed onto the stack. Engines are intended to be flexible and modular, allowing aspects of the simulation to be dropped and dragged as needed.

Engines can also be positioned as middleware to address both consistency (CON) and adaptivity (ADPT). To address adaptivity in our architecture, which required support for several notations, we introduced a middleware engine to produce agent behaviour at three moments, as shown in Figure 2a. In the future, any desired notation can be used in our simulation without rewriting the architecture, with the caveat that the simulation will hang until the agent produces a sequence of events for the stack. Furthermore, the middleware could be extended to address consistency, whereby internal states of each notation could be broadcast onto the event stack. Ensuring that observers could trace the agent's decision making and agree on a consistent truth between systems.

4.3 Events and Side Effects

Another key consideration was ensuring that events are visible to address tractable executions (TCT). Some events are for transitioning between internal states (IS), such as *agent-turn*, or phases of the simulation, such as *turn-end*. Emphasising visible events also helps with introspection (IPEC), as observers can navigate the history of a simulation without loss of information. However, communicating when data changes occur in the world state requires the specialisation of events to form a subclass called *side effects*.

[2]For the rest of the paper, we show names of events in the following manner: *[name]*.



(a) How our architecture produces event tape of a partial simulation result, where agents are given three opportunties to act during a turn.

(b) A demonstration of the evolution of an event stack, between two states $c \rightarrow d$, during a simulation, ordered from top to bottom.

Figure 2: Execution of our "Risk" simulation using event stacks and tapes. At runtime, the simulation's event stack grows and shrinks to represent the chain reaction between interactions, while an event tape records the simulation. Agents are given three moments to plan and then produce events, once for placement, attack, and movement.

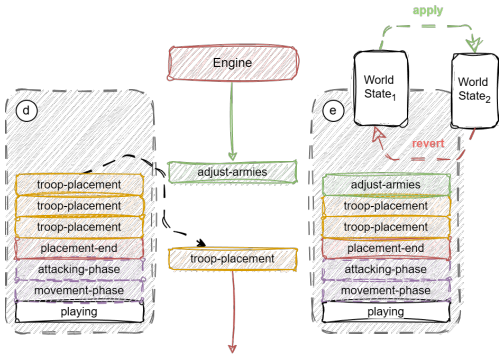


Figure 3: How the handling of a side effect (*adjust-armies*) occurs between two states d, e, to change the world state.

Side effects represent a privileged set of primitive behaviours that change the world state. These events are only producible via internal engines, i.e., agents cannot create new armies for themselves, and are processed by a single engine in our architecture. These events require visibility to ensure that observers can deterministically trigger changes and remain synchronised.

In Figure 3, we demonstrate how our architecture handles changing world state through *side effect* events like *adjust-armies*. When *adjust-armies* is consumed by the side-effect engine, the action is applied to the current world state to record a change to the world, i.e. moving from $\texttt{WS}_1 \rightarrow \texttt{WS}_2$. To allow for tractability, we assume that side effects have an `apply` function to transform a state into a new altered state, and a `revert` function to transition from an altered state back to the original state, e.g. moving from $\texttt{WS}_2 \rightarrow \texttt{WS}_1$. This way, when an event tape records a side effect, we can traverse both forward and backward directions without re-simulating to reach a desired state within the recording.

4.4 Risk Gameplay Engines

We now discuss the uses of stacks, tapes, engines, events, and side effects by briefly explaining our simulated game of Risk[3]. In the first reaction, *game* is popped off, and *playing* is placed on the stack. After which *playing* is popped off, and a new *playing* and *agent-turn* are placed on the stack. All these steps are handled by a single engine, which manages the win condition of the game, i.e. only one agent has armies and territories. If the condition does not hold, it will consume either *game* or *playing* to produce *playing* and/or *agent-turn* as needed. If *playing* is not placed on the stack, the chain reaction within the simulation ends.

Consuming *agent-turn* is another engine, which lays out the normal phases of a turn, which are *placement*, *attack*, and *movement*. This engine also consumes these produced events into their companion end events, e.g. it will consume *placement* to produce *placement-end*. Notably, when the engine consumes *movement*, it produces both *movement-end* and *agent-turn-end*; the latter event is eventually consumed to trigger the world state to move to the next player via a side effect called *advance-player*.

To arrive at the event stack d in Figure 2b, two engines react to *placement*, a system engine producing *placement-end* and the agent engine. The agent engine will trigger a planning phase for the autonomous agent, returning a plan of primitive behaviours (*troop-placement*, *attack*, and *move*). In this instance, the plan consisted of three *troop-placement* events, which indicate the intention of the agent to place troop/s in territories of the world. Note that at this stage, these are only intentions; they have not been acted upon or accepted by the simulator.

The transition from $d \rightarrow e$ in Figure 3 is controlled by a separate engine, which consumes *troop-placement* and produces either

[3] A board game about controlling territories and rolling dice to determine fights between contested territories, en.wikipedia.org/wiki/Risk_(game).

reject-placement or *adjust-armies*. If the intention of the troop placement is invalid, the engine returns *reject-placement* and the world stays the same. Otherwise, the troop placement is converted into a side effect (*adjust-armies*) event, and when popped off the stack, the world will change, reinforcing a territory with new troops. This explanation shows how interactions can be built using engines to modify the chain reaction occurring on the event stack during simulations.

5 Prototype Evaluation

To evaluate the efficiency of our architecture and validate that the proposed considerations are achievable, we built an MVP prototype[4] focused around the board game "Risk". Our prototype pitted several high-level agents against each other in a game to assess how notations could represent three strategies. We compare several implementations of each strategy across several notations, discussing their runtime performance and implementation implications.

5.1 Strategies

We implemented three different strategies for agents within our simulation of Risk: i) random, ii) defensive, and iii) aggressive.

The **random strategy** resolves decisions randomly. For placement, it places troops randomly on its owned territories. For attacking, it randomly picks an owned territory and randomly attacks an enemy adjacent territory. For movement, to avoid a stalled simulation with no winner, it randomly selects a safe[5] territory and moves its troops to a frontline[6] territory.

The **defensive strategy** aims to reduce the possibility of losing territories between turns. During placement, the agent places troops between their top n strongest frontlines[7]. For attacks, the strategy will only attack from the frontlines if they have at least three times as many troops or at least five troops more than the defender, whichever is higher. For movement, troops are evenly distributed across connected frontline territories.

The **aggressive strategy** aims to aggressively attack territories, seeking to breach the frontlines of other players. During placement, it places troops on a single frontline, which has the best attack opportunities. For attacks, the strategy attacks from the previous territory and follows up with attacks on adjacent territories, building a chain of attacks. The strategy assumes all attacks will be successful and half of the attacking troops will survive. For example, say we have four territories t_1, t_2, t_3, t_4, one chain might start attacking from t_1 to t_2, then from t_2 to t_3, and finally from t_3 to t_4. Each subsequent attack is issued with half minus one of the previous attacking troops, e.g., starting with 16 troops, then 7 troops, then 2 troops. For movement, troops are distributed based on the armies of adjacent enemy territories, across the n-strongest connected frontline territories.

For the selected notations, we modelled these three strategies using only primitive (Sec. 2.1.1) and compound (Sec. 2.1.2) behaviour to demonstrate the need for aggregated behaviour (Sec. 2.1.3). The potential use case of aggregated behaviour within our evaluation is having a better understanding of how attacks unfold, particularly for the aggressive strategy.

5.2 Experimental Setup

To evaluate the notations, we simulated games between seven agents (notations, including a hard-coded agent) across all possible permutations of the three strategies. Notably, DMN was not been included due to a lack of a third-party library. This resulted in a total of $3^7 = 2,187$ runs over 100 simulated turns, which took 18.82 wall hours, with each run averaging 11.74 minutes of CPU time. We report the average total runtime in seconds for planning placements, attacks, and movements for each notation. We also report an average score across runs, calculated based on the number of territories and troops at the end of each turn.

We also report on the implementation of strategies within each notation through two metrics. The first metric reports the number of elements (# in Table 2) needed to construct the strategy within the notation. For instance, we report the number of 'behaviours' or nodes within all trees of a pattern for behaviour trees, the number of models (atomic or coupled) for DEVS, and nodes within the net for DPNs and BPMNs. We also report on average the code complexity (Comp. in Table 2) of files that encoded the strategy using 'cyclomatic complexity'[8]. To avoid bias in implementation complexity, all implementations utilised a shared library to handle sensing and pathfinding tasks.

5.3 Results & Discussion

The results of our testing are shown in Table 2. It is important to note that the baseline complexity for each strategy is much higher than that of any of the evaluated notations. Ideally, by adding a conceptual layer, we can tighten the development's focus on behaviours, and the results showed a reduction in complexity (Comp.) across all selected notations. Notably, for the aggressive strategy, adding the conceptual layer reduced code complexity by roughly 50% across all notations over the baseline. However, these results should also take into account the number of constructs used, i.e. BT had a complexity of 1.91 but used 49 elements. As more constructs are added, code complexity decreases because internal paths are created rather than explicit paths within the source files.

When comparing the notations for planning time (runtime) to place events on the event stack, we see that notations with execution semantics outperformed the others, but all notations were slower than the baseline. The implementations for BT, HTN, and MCTS all experienced longer runtimes between simulations. However, the longer runtimes may be due to design choices by the software implementations, rather than stemming directly from the notation. The DEVS implementation had the least runtime between simulations, with BPMN and DPN as the second-best choices. The defensive strategy had longer runtimes, as it usually lasted the full 100 turns, increasing the average runtime.

We also report on the number of rejected events (as mentioned in Sec. 4.4) in Table 3. Rejections that can occur from the agent's planning consist of: *reject-placement*, *reject-attack*, and *reject-movement*. The number of observed rejections shows the difference between closed-loop actions, i.e. placement and movement, and open-ended

[4] github.com/adambanham/agent-risk .
[5] A territory surrounded by territories with the same owner.
[6] A territory with an adjacent territory owned by another player.
[7] n is decided upon randomly at runtime, but remains between 3 and 10.
[8] We used the Python software package 'radon' to compute complexity.

Table 2: The results of performance testing for strategies (Strat.) implemented in various notations.

	Strat.	#	Comp.	Runtime (std)	Score (std)
Base					
	rand.	n/a	4.4	0.06 (0.02)	51.67 (22.92)
	def.	n/a	6.6	0.18 (0.04)	132.21 (31.09)
	agg.	n/a	7.0	0.36 (0.13)	34.84 (15.21)
DEVS					
	rand.	16	2.06	0.56 (0.15)	58.78 (26.58)
	def.	32	2.10	1.74 (0.20)	140.70 (32.13)
	agg.	32	2.21	1.36 (0.22)	48.45 (14.87)
BT					
	rand.	30	1.71	16.48 (3.78)	55.29 (25.67)
	def.	35	1.81	56.76 (11.29)	135.66 (33.39)
	agg.	49	1.91	35.50 (15.56)	33.01 (13.81)
HTN					
	rand.	25	2.13	0.54 (0.14)	59.28 (27.70)
	def.	30	2.50	36.17 (15.70)	138.48 (33.16)
	agg.	34	2.20	13.44 (4.67)	29.77 (13.02)
BPMN					
	rand.	24	3.17	0.46 (0.10)	83.68 (36.56)
	def.	32	2.35	12.54 (2.01)	174.67 (35.70)
	agg.	28	2.45	5.38 (0.82)	67.01 (20.60)
DPN					
	rand.	10	3.52	0.39 (0.12)	63.51 (30.13)
	def.	14	3.05	5.28 (3.31)	159.14 (36.97)
	agg.	16	2.96	1.29 (0.45)	27.63 (11.70)
MCTS					
	rand.	6	2.29	29.91 (5.35)	73.97 (32.00)
	def.	8	2.16	25.16 (5.85)	158.83 (34.83)
	agg.	8	2.25	14.03 (1.95)	44.29 (11.22)

actions, attacks. For closed-loop actions, we expect zero rejections, i.e. the place and move columns in Table 3, as only implementation problems would lead to such rejections.

For attacks, we grouped rejections into three categories: attacks on owned territories (type-1 or T1), attacks with insufficient troops (type-2 or T2), and attacks requested from unowned territories (type-3 or T3). Type-1 rejections are likely to occur when several adjacent territories could attack the same territory; we see this rejection across all strategies. Type-2 rejections are either logic problems in the case of rand. and def. strategies, or due to the nature of chain attacks, which did not know the outcome of the last attack to inform the next attack within the agg. strategy. Type-3 rejections have similar causes to those of Type-2, but for the agg. strategy, they occur when the chain of attacks is unsuccessful at an early link, meaning the remaining attacks are all rejected. Whether these rejections occur depends to some extent on the simulation.

Table 3 demonstrates that without an interaction life-cycle as in aggregated behaviour, conditional and situational behaviours like the aggregated strategy require continuous replanning to avoid rejections. For modellers, this would mean creating a highly explicit plan, mapping out every possible event that could be seen and having a response. Not only would this dramatically increase development time, it would also greatly increase the number of branches needed to consider the behaviour's implementation.

Table 3: Breakdown of rejected events across simulation runs, grouped into placing, attacks, and moving troops.

	Strat.	place	attack			move
			T1	T2	T3	
Base						
	rand.	0	4140	0	0	0
	def.	0	49519	0	0	0
	agg.	0	97323	10290	44182	0
DEVS						
	rand.	0	6441	0	0	0
	def.	0	26058	0	0	0
	agg.	0	127524	19927	51811	0
BT						
	rand.	0	5111	0	0	0
	def.	0	39843	0	0	0
	agg.	0	0	21746	54034	0
HTN						
	rand.	0	4488	0	0	0
	def.	0	41808	0	0	0
	agg.	0	0	26318	138602	0
BPMN						
	rand.	0	10551	0	0	0
	def.	0	66092	0	0	0
	agg.	0	0	1483	271	0
DPN						
	rand.	0	5997	0	0	0
	def.	0	59685	0	0	0
	agg.	0	5809	7571	38710	0
MCTS						
	rand.	0	8398	0	0	0
	def.	0	0	0	0	0
	agg.	0	0	5851	18956	0

6 Conclusion

In this paper, we have considered many ways that modellers can explicitly and implicitly describe behaviour to study complex systems. We presented a synthesis of considerations for modellers and engineers working on large-scale, complex system simulators. In doing so, we compared several notations for describing behaviour in these systems and their capabilities for addressing our considerations. We proposed a novel event-driven software architecture to demonstrate the use cases of our considerations and their operationalisation. Finally, we evaluated both the practical implications of several notations and their performance within our novel architecture, with a focus on highlighting the need for aggregated behaviour as defined in Sec. 2.1.3.

In future work, we plan to expand our architecture to directly support aggregated behaviours while aligning with our considerations. Furthermore, we plan to explore additional simulation settings to ensure broader support for simulation studies and to generalise beyond military domains. Future work might consider how event stacks or tapes allow for the investigation of bisimilarity between the decision making of agents. Using simulations, techniques could automatically verify that new models conform to baselines, without requiring extensive testing and simulation to perform verification.

Acknowledgments

The Commonwealth of Australia (represented by the Department of Defence) supports this research through a Defence Science Partnerships agreement; Grant No.: 12870.

References

[1] Anurag Aggarwal, Mike Amend, Sylvain Astier, Alistair Barros, Rob Bartel, Mariano Benitez, Conrad Bock, Gary Brown, Justin Brunt, John Bulles, Martin Chapman, Fred Cummins, Rouven Day, Maged Elaasar, David Frankel, Denis Gagné, John Hall, Reiner Hille-Doering, Dave Ings, Pablo Irassar, Oliver Kieselbach, Matthias Kloppmann, Jana Koehler, Frank Michael Kraft, Tammo van Lessen, Frank Leymann, Antoine Lonjon, Sumeet Malhotra, Falko Menge, Jeff Mischkinsky, Dale Moberg, Alex Moffat, Ralf Mueller, Sjir Nijssen, Karsten Ploesser, Pete Rivett, Michael Rowley, Bernd Ruecker, Tom Rutt, Suzette Samoojh, Robert Shapiro, Vishal Saxena, Scott Schanel, Axel Scheithauer, Bruce Silver, Meera Srinivasan, Antoine Toulme, Ivana Trickovic, Hagen Voelzer, Franz Weber, Andrea Westerinen, and Stephen A. White. 2014. *BPMN™ — Business Process Model and Notation.* Standard. Object Management Group (OMG). https://www.omg.org/spec/BPMN Version 2.0.2.

[2] Mohamed A. Ahmed and Talal M. Alkhamis. 2009. Simulation optimization for an emergency department healthcare unit in Kuwait. *Eur. J. Oper. Res.* 198, 3 (2009), 936–942. doi:10.1016/J.EJOR.2008.10.025

[3] Alexander F. Arriaga, Angela M. Bader, Judith M. Wong, Stuart R. Lipsitz, William R. Berry, John E. Ziewacz, David L. Hepner, Daniel J. Boorman, Charles N. Pozner, Douglas S. Smink, and Atul A. Gawande. 2013. Simulation-based trial of surgical-crisis checklists. 368, 3 (2013), 246 – 253. doi:10.1056/NEJMsa1204720

[4] Ahmed Awad, Gero Decker, and Niels Lohmann. 2009. Diagnosing and Repairing Data Anomalies in Process Models. In *BPM Workshops (LNBIP, Vol. 43)*. Springer.

[5] Adam Banham, Arthur H. M. ter Hofstede, Sander J. J. Leemans, Felix Mannhardt, Robert Andrews, and Moe Thandar Wynn. 2024. Comparing Conformance Checking for Decision Mining: An Axiomatic Approach. *IEEE Access* 12 (2024).

[6] Jerry Banks. 2007. *Handbook of Simulation: Principles, Methodology, Advances, Applications, and Practice* (1 ed.). Wiley. 1–849 pages.

[7] Kimon Batoulis, Stephan Haarmann, and Mathias Weske. 2017. Various Notions of Soundness for Decision-Aware Business Processes. In *ER (LNCS, Vol. 10650)*. Springer.

[8] Kimon Batoulis, Andreas Meyer, Ekaterina Bazhenova, Gero Decker, and Mathias Weske. 2015. Extracting Decision Logic from Process Models. In *CAiSE (LNCS, Vol. 9097)*. Springer.

[9] Robert Joseph Bowen. 2008. *An Agent-Based Combat Simulation Framework in Support of Effect-Based and Net-Centric Evaluation.* Master's thesis. Computational Modeling & Simulation Engineering, Old Dominion University. doi:10.25777/6qzs-2n96

[10] Daniel S. Brown and Michael A. Goodrich. 2014. Limited Bandwidth Recognition of Collective Behaviors in Bio-Inspired Swarms. In *Proceedings of The 13th International Conference on Autonomous Agents and Multiagent Systems*. Association for Computing Machinery, Inc, New York, USA, 405 – 412.

[11] Cameron Browne, Edward Jack Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavener, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. 2012. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games* 4, 1 (2012), 1–43. doi:10.1109/TCIAIG.2012.2186810

[12] Josep Carmona, Boudewijn F. van Dongen, Andreas Solti, and Matthias Weidlich. 2018. *Conformance Checking - Relating Processes and Models.* Springer. doi:10.1007/978-3-319-99414-7

[13] R. Castro, V. Shemeshenko, and P. Mira. 2025. A Novel Integrated Agent-Based Computational Economics and Multi-Agent Reinforcement Learning Framework: Application to Macrocrisis Phenomena. In *Resilient Complex Adaptive Systems: Modeling, Simulation, Visualization and Engineering Approaches*, C. Szabo, R. Castro, S. Mittal, and V. Shemeshenko (Eds.). John Wiley & Sons, Hoboken, NJ.

[14] David Chapela-Campa, Orlenys López-Pintado, Ihar Suvorau, and Marlon Dumas. 2025. SIMOD: Automated discovery of business process simulation models. *SoftwareX* 30 (2025), 102157. doi:10.1016/J.SOFTX.2025.102157

[15] C. Chen, S. B. Nagl, and C. D. Clack. 2007. Specifying, Detecting and Analysing Emergent Behaviours in Multi-Level Agent-Based Simulations. In *Proceedings of the Summer Computer Simulation Conference*. Association for Computing Machinery, Inc, New York, USA, 969–976.

[16] Alex Chung Hen Chow and Bernard P. Zeigler. 1994. Parallel DEVS: a parallel, hierarchical, modular, modeling formalism. In *Proceedings of the 26th conference on Winter simulation, WSC 1994, Lake Buena Vista, FL, USA, December 11-14, 1994*, Deborah A. Sadowski, Andrew F. Seila, Mani S. Manivannan, and Jeffrey D. Tew (Eds.). ACM, 716–722. doi:10.1109/WSC.1994.717419

[17] Michele Colledanchise and Petter Ögren. 2017. How Behavior Trees Modularize Hybrid Control Systems and Generalize Sequential Behavior Compositions, the Subsumption Architecture, and Decision Trees. *IEEE Trans. Robotics* 33, 2 (2017), 372–389. doi:10.1109/TRO.2016.2633567

[18] Steve Cook, Conrad Bock, Pete Rivett, Tom Rutt, Ed Seidewitz, Bran Selic, and Doug Tolbert. 2017. *Unified Modeling Language.* Standard. Object Management Group (OMG). https://www.omg.org/spec/UML Version 2.5.1.

[19] Jason P. Davis, Kathleen M. Eisenhardt, and Christopher B. Bingham. 2007. Developing Theory Through Simulation Methods. *Academy of Management Review* 32, 2 (2007), 480–499.

[20] Massimiliano de Leoni, Paolo Felli, and Marco Montali. 2018. A Holistic Approach for Soundness Verification of Decision-Aware Process Models. In *ER (LNCS, Vol. 11157)*. Springer.

[21] Massimiliano de Leoni, Paolo Felli, and Marco Montali. 2021. Integrating BPMN and DMN: Modeling and Analysis. *J. Data Semant.* 10, 1-2 (2021).

[22] Johannes De Smedt, Faruk Hasic, Seppe K. L. M. vanden Broucke, and Jan Vanthienen. 2017. Towards a Holistic Discovery of Decisions in Process-Aware Information Systems. In *BPM (LNCS, Vol. 10445)*. Springer.

[23] Johannes De Smedt, Faruk Hasic, Seppe K. L. M. vanden Broucke, and Jan Vanthienen. 2019. Holistic discovery of decision models from process execution data. *Knowl. Based Syst.* 183 (2019).

[24] JL Deneubourg and S Goss. 1989-12. Collective patterns and decision-making. *Ethology, ecology & evolution.* 1, 4 (1989-12), 295 – 311.

[25] Remco M. Dijkman. 2024. SimPN: A Python Library for Modeling and Simulating Timed, Colored Petri Nets. In *Proceedings of the Best Dissertation Award, Doctoral Consortium, and Demonstration & Resources Forum at BPM 2024 co-located with 22nd International Conference on Business Process Management (BPM 2024), Krakow, Poland, September 1st to 6th, 2024 (CEUR Workshop Proceedings, Vol. 3758)*. 71–75. https://ceur-ws.org/Vol-3758/paper-12.pdf

[26] Remco M. Dijkman, Marlon Dumas, and Chun Ouyang. 2008. Semantics and analysis of business process models in BPMN. *Inf. Softw. Technol.* 50, 12 (2008), 1281–1294. doi:10.1016/J.INFSOF.2008.02.006

[27] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. 2018. *Fundamentals of Business Process Management, Second Edition.* Springer. doi:10.1007/978-3-662-56509-4

[28] Paolo Felli, Massimiliano de Leoni, and Marco Montali. 2021. Soundness Verification of Data-Aware Process Models with Variable-to-Variable Conditions. *Fundam. Informaticae* 182, 1 (2021).

[29] Paolo Felli, Marco Montali, and Sarah Winkler. 2022. Soundness of Data-Aware Processes with Arithmetic Conditions. In *CAiSE (LNCS, Vol. 13295)*. Springer.

[30] Paolo Felli, Marco Montali, and Sarah Winkler. 2023. Repairing Soundness Properties in Data-Aware Processes. In *ICPM*. IEEE.

[31] Bruce W. Fowler. 1996. De Physica Belli – An Introduction to Lanchestrial Attrition Mechanics. *Simulation and Tactical Technology Information Analysis Center* (1996).

[32] Jakob Freund and Bernd Rücker. 2019. *Real-Life BPMN* (4 ed.). Camunda.

[33] Ilche Georgievski and Marco Aiello. 2015. HTN planning: Overview, comparison, and beyond. *Artif. Intell.* 222 (2015), 124–156. doi:10.1016/J.ARTINT.2015.02.002

[34] Malik Ghallab, Dana Nau, and Paolo Traverso. 2025. *Acting, planning, and learning.* Cambridge University Press, Cambridge ; New York, NY.

[35] Jacques Gignoux, Guillaume Chérel, Ian D Davies, Shayne R Flint, and Eric Lateltin. 2017. Emergence and complex systems: The contribution of dynamic graph theory. *Ecological Complexity* 31 (2017), 34–49.

[36] Alexandre Goossens, Johannes De Smedt, and Jan Vanthienen. 2024. Extracting process-aware decision models from object-centric process data. *Inf. Sci.* 682 (2024).

[37] Bradley Hayes and Brian Scassellati. 2016. Autonomously constructing hierarchical task networks for planning and human-robot collaboration. In *2016 IEEE International Conference on Robotics and Automation, ICRA 2016, Stockholm, Sweden, May 16-21, 2016*. IEEE, 5469–5476. doi:10.1109/ICRA.2016.7487760

[38] Carl-Johan Hoel, Katherine Rose Driggs-Campbell, Krister Wolff, Leo Laine, and Mykel J. Kochenderfer. 2020. Combining Planning and Deep Reinforcement Learning in Tactical Decision Making for Autonomous Driving. *IEEE Trans. Intell. Veh.* 5, 2 (2020), 294–305. doi:10.1109/TIV.2019.2955905

[39] John H Holland. 2006. Studying Complex Adaptive Systems. *Journal of Systems Science and Complexity* 19, 1 (2006), 1–8.

[40] Matteo Iovino, Edvards Scukins, Jonathan Styrud, Petter Ögren, and Christian Smith. 2022. A survey of Behavior Trees in robotics and AI. *Robotics Auton. Syst.* 154 (2022), 104096. doi:10.1016/J.ROBOT.2022.104096

[41] David R.C. Hill Jean-Baptiste Filippi, Teruhisa Komatsu. 2018. *Discrete-Event Modeling and Simulation: Theory and Applications.* CRC Press, Chapter Environmental Models in DEVS Different Approaches for Different Applications.

[42] Kurt Jensen and Lars Michael Kristensen. 2009. *Coloured Petri Nets - Modelling and Validation of Concurrent Systems.* Springer. doi:10.1007/B95112

[43] Anja Johansson and Pierangelo Dell'Acqua. 2012. Emotional behavior trees. In *2012 IEEE Conference on Computational Intelligence and Games, CIG 2012, Granada, Spain, September 11-14, 2012*. IEEE, 355–362. doi:10.1109/CIG.2012.6374177

[44] A. Kubik. 2003. Towards a Formalization of Emergence. *Journal of Artificial Life* 9(1) (2003), 41–65. Issue 1.

[45] Karl Kurzer, Chenyang Zhou, and Johann Marius Zöllner. 2018. Decentralized Cooperative Planning for Automated Vehicles with Hierarchical Monte Carlo Tree Search. In *2018 IEEE Intelligent Vehicles Symposium, IV 2018, Changshu, Suzhou, China, June 26-30, 2018*. IEEE, 529–536. doi:10.1109/IVS.2018.8500712

[46] F. W. Lanchester. 1916. *Aircraft in warfare : the dawn of the Fourth Arm*. Constable, London.

[47] David C. Lane, Camilla Monefeldt, and Jonathan Rosenhead. 2000. Looking in the wrong place for healthcare improvements: A system dynamics study of an accident and emergency department. *J. Oper. Res. Soc.* 51, 5 (2000), 518–531. doi:10.1057/PALGRAVE.JORS.2600892

[48] Chi G. Lee and Sang C. Park. 2014. Survey on the virtual commissioning of manufacturing systems. *J. Comput. Des. Eng.* 1, 3 (2014), 213–222. doi:10.7315/JCDE.2014.021

[49] Sam Leewis, Koen Smit, Bas van den Boom, and Johan Versendaal. 2025. Improving operational decision-making through decision mining - utilizing method engineering for the creation of a decision mining method. *Inf. Softw. Technol.* 179 (2025).

[50] Azumah Mamudu, Wasana Bandara, Moe Thandar Wynn, and Sander J. J. Leemans. 2025. Process Mining Success Factors and Their Interrelationships. *Bus. Inf. Syst. Eng.* 67, 3 (2025), 389–408. doi:10.1007/S12599-024-00860-Z

[51] Leandro Soriano Marcolino and Hitoshi Matsubara. 2011. Multi-agent Monte Carlo Go. In *10th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2011), Taipei, Taiwan, May 2-6, 2011, Volume 1-3*. IFAAMAS, 21–28. http://portal.acm.org/citation.cfm?id=2030474&CFID=69153967&CFTOKEN=38069692

[52] Alejandro Marzinotto, Michele Colledanchise, Christian Smith, and Petter Ögren. 2014. Towards a unified behavior trees framework for robot control. In *2014 IEEE International Conference on Robotics and Automation, ICRA 2014, Hong Kong, China, May 31 - June 7, 2014*. IEEE, 5420–5427. doi:10.1109/ICRA.2014.6907656

[53] Jan Mendling, Hajo A. Reijers, and Wil M. P. van der Aalst. 2010. Seven process modeling guidelines (7PMG). *Inf. Softw. Technol.* 52, 2 (2010), 127–136. doi:10.1016/J.INFSOF.2009.08.004

[54] Falko Menge, Alan Fish, Alessandra Bagnato, Denis Gagne, Elisa Kendall, Gil Segal, J.D. Baker, James Taylor, Jan Vanthienen, Octavian Patrascoiu, Serge Schiltz, Sjir Nijssen, Stephen White, Tibor Zimanyi, Uwe Kaufmann, and Zbigniew Misiak. 2024. *DMN™ — Decision Model and Notation*. Standard. Object Management Group (OMG). https://www.omg.org/spec/DMN Version 1.6.

[55] S. Mittal. 2013. Emergence in Stigmergic and Complex Adaptive Systems: A Formal Discrete Event Systems Perspective. *Cognitive Systems Research* 21(1) (2013), 22–39.

[56] Saurabh Mittal and Larry Rainey. 2015. Harnessing Emergence: The Control and Design of Emergent Behavior in System of Systems Engineering. In *Proceedings of the Conference on Summer Computer Simulation* (Chicago, Illinois) *(SummerSim '15)*. Society for Computer Simulation International, San Diego, CA, USA, 1–10. http://dl.acm.org/citation.cfm?id=2874916.2874979

[57] Saurabh Mittal and José L. Risco-Martín. 2017. *Simulation-Based Complex Adaptive Systems*. Springer International Publishing, Cham, 127–150. doi:10.1007/978-3-319-61264-5_6

[58] Saurabh Mittal and Andreas Tolk. 2019. *Complexity challenges in cyber physical systems: Using modeling and simulation (M&S) to support intelligence, adaptation and autonomy*. John Wiley & Sons.

[59] Matthew Molineaux, Matthew Klenk, and David W. Aha. 2010. Goal-Driven Autonomy in a Navy Strategy Simulation. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2010, Atlanta, Georgia, USA, July 11-15, 2010*. AAAI Press, 1548–1554. doi:10.1609/AAAI.V24I1.7576

[60] Dana Nau, Tsz-Chiu Au, Okhtay Ilghami, Ugur Kuter, J. William Murdock, Dan Wu, and Fusun Yaman. 2003. SHOP2: An HTN planning system. *Journal of Artificial Intelligence Research* 20 (2003), 379 – 404. doi:10.1613/jair.1141

[61] Muaz AK Niazi. 2013. Complex Adaptive Systems Modeling: A Multidisciplinary Roadmap. *Complex Adaptive Systems Modeling* 1, 1 (2013), 1–14.

[62] Michael J North, Nicholson T Collier, Jonathan Ozik, Eric R Tatara, Charles M Macal, Mark Bragen, and Pam Sydelko. 2013. Complex Adaptive Systems Modeling With Repast Simphony. *Complex Adaptive Systems Modeling* 1, 1 (2013), 15–23.

[63] Lewis Ntaimo, Xiaolin Hu, and Yi Sun. 2008. DEVS-FIRE: Towards an Integrated Simulation Environment for Surface Wildfire Spread and Containment. *Simulation* 84, 4 (2008), 137 – 155. doi:10.1177/0037549708094047

[64] James Nutaro, Phani Teja Kuruganti, Laurie Miller, Sara Mullen, and Mallikarjun Shankar. 2007. Integrated Hybrid-Simulation of Electric Power and Communications Systems. In *2007 IEEE Power Engineering Society General Meeting*. 1–8. doi:10.1109/PES.2007.386202

[65] Santiago Ontañón and Michael Buro. 2015. Adversarial Hierarchical-Task Network Planning for Complex Real-Time Games. In *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*. AAAI Press, 1652–1658. http://ijcai.org/Abstract/15/236

[66] Baris Özkan, Marijn Koops, Oktay Türetken, and Hajo A. Reijers. 2024. The Influence of Business Process Management System Implementation on an Organization's Process Orientation: A Case Study of a Financial Service Provider. *Inf. Syst. Manag.* 41, 4 (2024), 377–398. doi:10.1080/10580530.2023.2286980

[67] Ö Özmen, J Smith, and L Yilmaz. 2013. An Agent-based Simulation Study Of A Complex Adaptive Collaboration Network. In *Proceedings of the Winter Simulation Conference*, Raghu Pasupathy, Seong-He Kim, and Andreas Tolk (Eds.). Institute of Electrical and Electronics Engineers, Inc Press, Piscataway, NJ, USA, 412–423.

[68] Julia K. Parrish, Steven V. Viscido, and Daniel Grünbaum. 2002. Self-organized fish schools: An examination of emergent properties. *Biological Bulletin* 202, 3 (2002), 296 – 305. doi:10.2307/1543482

[69] Diego Pérez-Liébana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius, and Simon M. Lucas. 2019. General Video Game AI: A Multitrack Framework for Evaluating Agents, Games, and Content Generation Algorithms. *IEEE Trans. Games* 11, 3 (2019), 195–214. doi:10.1109/TG.2019.2901021

[70] Gonzalo Flórez Puga, Marco Antonio Gómez-Martín, Pedro Pablo Gómez-Martín, Belén Díaz-Agudo, and Pedro A. González-Calero. 2009. Query-Enabled Behavior Trees. *IEEE Trans. Comput. Intell. AI Games* 1, 4 (2009), 298–308. doi:10.1109/TCIAIG.2009.2036369

[71] Steve Rabin. 2014. *Game AI pro : collected wisdom of game AI professionals*. CRC Press, Taylor & Francis Group, Boca Raton.

[72] Craig W. Reynolds. 1987. Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1987, Anaheim, California, USA, July 27-31, 1987*. ACM, 25–34. doi:10.1145/37401.37406

[73] José L. Risco-Martín, Saurabh Mittal, Kevin Henares, Román Cárdenas, and Patricia Arroba. 2023. xDEVS: A toolkit for interoperable modeling and simulation of formal discrete event systems. *Softw. Pract. Exp.* 53, 3 (2023), 748–789. doi:10.1002/SPE.3168

[74] Bernd Rücker and Leon Strauch. 2025. *Enterprise Process Orchestration : A Hands-On Guide to Strategy, People, and Technology That Will Transform Your Business* (1 ed.). Wiley, Hoboken, NJ.

[75] Earl D. Sacerdoti. 1975. The Nonlinear Nature of Plans. In *Advance Papers of the Fourth International Joint Conference on Artificial Intelligence, Tbilisi, Georgia, USSR, September 3-8, 1975*. 206–214. http://ijcai.org/Proceedings/75/Papers/028.pdf

[76] Marwin H. S. Segler, Mike Preuss, and Mark P. Waller. 2018. Planning chemical syntheses with deep neural networks and symbolic AI. *Nat.* 555, 7698 (2018), 604–610. doi:10.1038/NATURE25978

[77] A. K. Seth. 2008. Measuring Emergence via Nonlinear Granger Causality. In *Proceedings of the Eleventh International Conference on the Simulation and Synthesis of Living Systems*. Institute of Electrical and Electronics Engineers, Inc, Piscataway, NJ, USA, 545–553.

[78] Alexander Shoulson, Francisco M. Garcia, Matthew Jones, Robert Mead, and Norman I. Badler. 2011. Parameterizing Behavior Trees. In *Motion in Games*. Springer Berlin Heidelberg, Berlin, Heidelberg, 144–155.

[79] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy P. Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. 2016. Mastering the game of Go with deep neural networks and tree search. *Nat.* 529, 7587 (2016), 484–489. doi:10.1038/NATURE16961

[80] Stephen J. J. Smith, Dana S. Nau, and Thomas A. Throop. 1998. Success in Spades: Using AI Planning Techniques to Win the World Championship of Computer Bridge. In *Proceedings of the Fifteenth National Conference on Artificial Intelligence and Tenth Innovative Applications of Artificial Intelligence Conference, AAAI 98, IAAI 98, July 26-30, 1998, Madison, Wisconsin, USA*. AAAI Press / The MIT Press, 1079–1086. http://www.aaai.org/Library/IAAI/1998/iaai98-010.php

[81] David T. Sturrock. 2015. Tutorial: tips for successful practice of simulation. In *Proceedings of the 2015 Winter Simulation Conference, Huntington Beach, CA, USA, December 6-9, 2015*. IEEE/ACM, 1756–1764. doi:10.1109/WSC.2015.7408293

[82] D.J.T. Sumpter. 2006. The principles of collective animal behaviour. *Philosophical Transactions of the Royal Society B: Biological Sciences* 361, 1465 (2006), 5 – 22. doi:10.1098/rstb.2005.1733

[83] Maciej Swiechowski, Konrad Godlewski, Bartosz Sawicki, and Jacek Mandziuk. 2023. Monte Carlo Tree Search: a review of recent modifications and applications. *Artif. Intell. Rev.* 56, 3 (2023), 2497–2562. doi:10.1007/S10462-022-10228-Y

[84] Claudia Szabo. 2024. Complex Systems Modeling and Analysis. In *Winter Simulation Conference, WSC 2024, Orlando, FL, USA, December 15-18, 2024*. IEEE, 1–14. doi:10.1109/WSC63780.2024.10838891

[85] Claudia Szabo, Robin Baker, Glen Pearce, Eyoel Teffera, and Anthony Perry. 2025. Overview and Challenges of Distributed Decision Making in Resource Contested and Dynamic Environments. *ACM Comput. Surv.* 57, 9 (2025), 235:1–235:34. doi:10.1145/3719001

[86] Claudia Szabo, Rodrigo Castro, Joachim Denil, and Susan M Sanchez. 2023. Resilience and Complexity in Socio-Cyber-Physical Systems. In *2023 Winter*

Simulation Conference (WSC). IEEE, 660–670.

[87] Claudia Szabo, Joshua Groot, Thomas McAtee, and Christo Pyromallis. 2019. Who Did This: Identifying Causes of Emergent Behavior in Complex Systems. In *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 262–270.

[88] Claudia Szabo, Joshua Groot, Thomas McAtee, and Christo Pyromallis. 2019. Who Did This: Identifying Causes of Emergent Behavior in Complex Systems. In *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 262–270.

[89] C. Szabo and Y.M. Teo. 2012. An Integrated Approach for the Validation of Emergence in Component-based Simulation Models. In *Proceedings of the Winter Simulation Conference*, Christoph Laroque, Raghu Pasupathy, and Jan Himmelspach (Eds.). Institute of Electrical and Electronics Engineers, Inc Press, Piscataway, NJ, USA, 2412–2423.

[90] C. Szabo and Y. M. Teo. 2006. *An Approach to Syntactic Composability and Model Reuse in Composable Simulation*. Technical Report. Department of Computer Science, National University of Singapore.

[91] Claudia Szabo and Yong Meng Teo. 2013. Post-mortem Analysis of Emergent Behavior in Complex Simulation Models. In *Proceedings of the 2013 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation*. Association for Computing Machinery, Inc, New York, USA, 241–252.

[92] A. Tolk. 2006. What Comes After the Semantic Web - PADS Implications for the Dynamic Web. In *Proceedings of the 20th Workshop on Principles of Advanced and Distributed Simulation*. Singapore, 55–62.

[93] Andreas Tolk. 2012. *Engineering principles of combat modeling and distributed simulation*. Wiley, Hoboken.

[94] Andreas Tolk. 2024. Conceptual alignment for simulation interoperability: lessons learned from 30 years of interoperability research. *Simul.* 100, 7 (2024), 709–726. doi:10.1177/00375497231216471

[95] A. M. Uhrmacher, D. Degenring, J. Lemcke, and M. Krahmer. 2004. Towards Reusing Model Components in Systems Biology. In *Proceedings of the Computational Methods for Systems Biology Conference*. 192–206.

[96] Adelinde M. Uhrmacher, Peter I. Frazier, Reiner Hähnle, Franziska Klügl, Fabian Lorig, Bertram Ludäscher, Laura Nenzi, Cristina Ruiz Martin, Bernhard Rumpe, Claudia Szabo, Gabriel A. Wainer, and Pia Wilsdorf. 2024. Context, Composition, Automation, and Communication: The C^2AC Roadmap for Modeling and Simulation. *ACM Trans. Model. Comput. Simul.* 34, 4 (2024), 23:1–23:51. doi:10.1145/3673226

[97] Adelinde M. Uhrmacher and Danny Weyns (Eds.). 2009. *Multi-Agent Systems - Simulation and Applications*. CRC Press / Taylor & Francis. doi:10.1201/9781420070248

[98] Wil M.P. van der Aalst, Arthur H. M. ter Hofstede, Bartek Kiepuszewski, and Alistair P. Barros. 2003. Workflow Patterns. *Distributed Parallel Databases* 14, 1 (2003), 5–51. doi:10.1023/A:1022883727209

[99] Wil M. P. van der Aalst. 2015. Business Process Simulation Survival Guide. In *Handbook on Business Process Management 1, Introduction, Methods, and Information Systems, 2nd Ed.* Springer, 337–370. doi:10.1007/978-3-642-45100-3_15

[100] Wil M. P. van der Aalst. 2016. *Process Mining - Data Science in Action, Second Edition*. Springer.

[101] Gabriel Wainer. 2006. Applying Cell-DEVS Methodology for Modeling the Environment. *Simulation* 82, 10 (2006), 635 – 660. doi:10.1177/0037549706073698

[102] Gabriel A. Wainer, Pieter J. Mosterman, Gabriel A Wainer, and Pieter J Mosterman. 2018. *Discrete-Event Modeling and Simulation: Theory and Applications* (1 ed.). CRC Press, United States.

[103] Moe Thandar Wynn. 2006. *Semantics, verification, and implementation of workflows with cancellation regions and OR-joins*. Ph. D. Dissertation. Queensland University of Technology. https://eprints.qut.edu.au/16324/

[104] Moe T. Wynn, Wil M.P. van der Aalst, Eric Verbeek, Estefania Serral Asensio, Claudio Di Ciccio, Rajesh Murth, Peter Blank, Philipp Herrmann, Arif Selcuk Ogrenci, Marcus Brenscheidt, Nancy Landreville, Andreea Ungureanu, Sergio Correia, and Julian Lebherz. 2023. IEEE Standard for eXtensible Event Stream (XES) for Achieving Interoperability in Event Logs and Event Streams. *IEEE Std 1849-2023 (Revision of IEEE Std 1849-2016)* (2023), 1–55. doi:10.1109/IEEESTD.2023.10267858

[105] Xiao Xu, Mei Yang, and Ge Li. 2018. Adaptive CGF Commander Behavior Modeling Through HTN Guided Monte Carlo Tree Search. *J. Syst. Sci. Syst. Eng.* 27 (2018), 231––249. doi:10.1007/s11518-018-5366-8

[106] Matteo Zavatteri, Davide Bresolin, Massimiliano de Leoni, and Aurelo Makaj. 2025. Data-aware process models: From soundness checking to repair. *Data Knowl. Eng.* 155 (2025).

[107] Bernard P. Zeigler, Alexandre Muzy, and Ernesto Kofman. 2019. *Theory of modeling and simulation : discrete event and iterative system computational foundations* (third edition ed.). Academic Press, London.